

# Enhanced Bug Report Classification Using XGBoost and TF-IDF

Abdelmalek Maskri

2541248

## 1 Introduction

The classification of bug reports is vital for maintaining large software projects, especially in deep learning frameworks such as TensorFlow, PyTorch, and Keras, where identifying performance-related bugs is challenging due to technical complexity. This project aims to classify bug reports as performance-related or not, addressing framework complexity, resource allocation, and domain-specific language processing.

I have chosen Coursework 1: Tool Building Project, focusing on Lab 1: Bug Report Classification. Motivated by my interest in classification models from previous Machine Learning and NLP modules, I aim to address two key limitations of the baseline approach: its difficulty handling semantic relationships in text, and the class imbalance present in the dataset (only 16.4% of reports are performance-related).

This project proposes an enhanced classification approach to improve accuracy and reliability. By leveraging advanced NLP techniques, The resulting tool aims to help developers more effectively detect performance-related bugs, a crucial step in maintaining and optimizing deep learning frameworks.

## 2 Related Work

Bug report classification has evolved from basic information retrieval methods to advanced machine learning and deep learning approaches. Traditional models such as Naive Bayes and SVM have been widely used in early work, often in combination with TF-IDF for text representation [1, 2]. These methods provided a baseline for categorizing bug reports but struggled with complex semantics and class imbalance.

Recent advances leverage ensemble methods like XGBoost, which offer improved performance due to their ability to model non-linear relationships and manage imbalanced datasets [3]. TF-IDF remains effective for representing bug report text, especially when computational resources are limited [4]. Deep learning models, including CNNs and LSTMs, have also been proposed for classifying bug severity and types, but often require large datasets and more training time [5, 6].

Specifically for deep learning frameworks, performance-related bugs are challenging due to their dependency on hardware and model configuration. Studies like those by Long and Chen [7] highlight that many performance bug reports are mislabeled or invalid, motivating better classification tools. Recent work incorporating pretrained language models and knowledge-augmented techniques has significantly boosted classification accuracy for such tasks [8].

## 3 Solution

The proposed solution enhances bug report classification through a combination of advanced text processing techniques and the XGBoost classifier. The solution addresses several limitations of the baseline Naive Bayes approach:

### 3.1 Text Processing Pipeline

The text processing pipeline includes several steps designed to improve the quality of features extracted from bug reports:

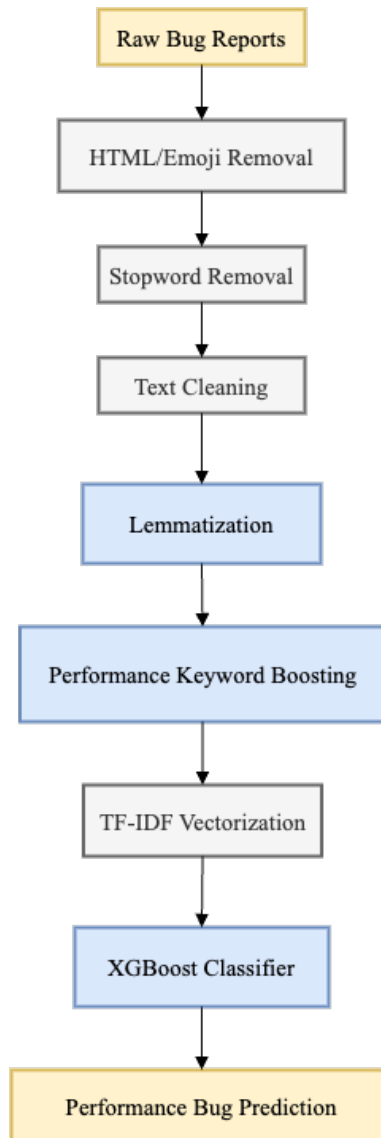


Figure 1: Pipeline for Bug Report Classification from Raw Text to Prediction

1. **HTML and Emoji Removal:** Bug reports often contain HTML tags and emojis, which add noise to the text. Regular expressions are used to remove these elements.
2. **Stopword Removal:** Common English stopwords are removed to focus on meaningful content words.
3. **Text Cleaning:** Non-alphanumeric characters are removed, and the text is converted to lowercase to standardize the format.
4. **Lemmatization:** Words are normalized using lemmatization, which reduces words to their base or dictionary form (known as a lemma). This process is linguistically accurate and ensures the resulting words maintain their semantic meaning, unlike more aggressive approaches that might distort meaning.

---

```
nltk.download('wordnet')
nltk.download('omw-1.4')

lemmatizer = WordNetLemmatizer()

def apply_lemmatization(text):
    """Lemmatize each word in the text."""
    return ' '.join([lemmatizer.lemmatize(word, pos=wordnet.VERB) for word in
        ↪ text.split()])
```

---

5. **Performance Keyword Boosting:** A novel technique is implemented to boost the importance of performance-related terms in the TF-IDF vectorization. This involves identifying a predefined list of performance-related keywords (such as “slow,” “speed,” “memory,” “cpu,” “performance”) and repeating these terms in the text to increase their TF-IDF weight when they appear in a report.

---

```
performance_terms = ['slow', 'speed', 'fast', 'memory', 'cpu', 'gpu', 'performance',
                    'latency', 'throughput', 'bottleneck', 'optimization', 'efficient',
                    'regression', 'benchmark', 'overhead', 'usage']

def boost_performance_keywords(text):
    """Repeat performance-related keywords to boost their importance"""
    for term in performance_terms:
        if term in text.lower():
            # Repeat the term to increase its TF-IDF weight
            text = text + " " + term + " " + term
    return text
```

---

## 3.2 Classification Using XGBoost

XGBoost (eXtreme Gradient Boosting) is chosen as the classifier for several reasons:

1. **Handling Non-linear Relationships:** Unlike Naive Bayes, XGBoost can capture complex, non-linear relationships between features.
2. **Robustness to Overfitting:** XGBoost includes regularization terms in its objective function, making it less prone to overfitting.
3. **Feature Importance:** XGBoost provides feature importance scores, which can be useful for understanding which terms are most predictive of performance bugs.
4. **Handling Class Imbalance:** XGBoost allows for class weighting, which is essential given the imbalanced nature of the dataset (only about 16.4% of reports are performance-related).

The XGBoost classifier is configured with the following parameters:

---

```
clf = XGBClassifier(
    learning_rate=0.1,
    max_depth=3,
    n_estimators=100,
    scale_pos_weight=5, # Give more weight to positive class
    eval_metric='logloss',
    random_state=42
)
```

---

The `scale_pos_weight` parameter is set to 5 to account for the class imbalance, giving more weight to the minority class (performance-related bugs). The `max_depth` is limited to 3 to prevent overfitting, and 100 trees (`n_estimators`) are used in the ensemble.

## 3.3 Design Rationale

Several key design decisions were made in developing this solution:

1. **Why XGBoost over Deep Learning?:** While deep learning approaches can potentially achieve higher accuracy, they require large amounts of data and computational resources. XGBoost provides a good balance between performance and efficiency, making it suitable for this task.
2. **Performance Keyword Boosting:** This technique was developed based on the observation that performance bugs often contain specific keywords related to system performance. By boosting these terms, we emphasize domain-specific knowledge in the classification process.

3. **Lemmatization:** The choice to use lemmatization instead of other normalization techniques was deliberate as it reduces words to their dictionary form while preserving meaning. This linguistically accurate approach is beneficial for technical text where precise meaning is important for classification accuracy.
4. **Class Weighting:** The decision to use class weighting instead of undersampling or oversampling was made to preserve all available training data while still addressing the class imbalance.

These design decisions work together to create a solution that is specifically tailored to the bug report classification task in deep learning frameworks.

I experimented with different embedding methods like BERT, Word2Vec, and ELMO, but they didn't necessarily provide better results than TF-IDF. One key reason is the nature of the task TF-IDF works well since bug reports rely heavily on term frequency rather than deep contextual relationships. We also don't have a large dataset to train deep models effectively, and computational resources are limited. Methods like ELMO required excessive training time and massive hardware support, making them impractical. For simplicity, efficiency, and performance trade-off, TF-IDF was the best choice.

## 4 Setup

### 4.1 Datasets

The experiments were conducted on bug report datasets from five popular deep learning frameworks:

1. **TensorFlow:** 1,490 reports (18.7% performance-related)
2. **PyTorch:** 752 reports (12.6% performance-related)
3. **Keras:** 668 reports (20.2% performance-related)
4. **MXNet:** 516 reports (12.6% performance-related)
5. **Caffe:** 286 reports (11.5% performance-related)

In total, the dataset contains 3,712 GitHub reports, with 607 (16.4%) being performance-related.

### 4.2 Experimental Procedure

The experimental procedure follows these steps:

1. **Data Preparation:** For each project, the title and body of bug reports were merged to create a single text field for analysis.
2. **Text Processing:** The text processing pipeline described in Section 3.1 was applied to all reports.
3. **Train-Test Split:** For each experiment, the data was randomly split into 80% training and 20% testing sets, with stratification to maintain the same class distribution in both sets.
4. **Feature Extraction:** TF-IDF vectorization was applied to convert the processed text into numerical features, with a maximum of 1,000 features.
5. **Model Training:** The XGBoost classifier was trained on the training set.
6. **Evaluation:** The model was evaluated on the test set using multiple metrics.
7. **Repetition:** Steps 3-6 were repeated 10 times with different random seeds to ensure robust results.

### 4.3 Evaluation Metrics

The following metrics were used to evaluate the performance of the models:

1. **Accuracy:** The ratio of correctly predicted observations to the total observations.
2. **Precision:** The ratio of correctly predicted positive observations to the total predicted positives, measuring how many identified performance bugs are actually performance-related.
3. **Recall:** The ratio of correctly predicted positive observations to all actual positives, measuring the ability to find all performance-related bugs.
4. **F1 Score:** The weighted average of Precision and Recall, particularly useful for imbalanced datasets.
5. **AUC (Area Under the ROC Curve):** Measures the model's ability to distinguish between classes across all possible classification thresholds, with 1.0 representing perfect classification.

All metrics were calculated using the macro-averaging method to give equal weight to both classes despite the class imbalance.

### 4.4 Baseline Comparison

The proposed solution was compared against the baseline approach of Naive Bayes with TF-IDF vectorization, as specified in the coursework. The baseline was implemented as provided in the lab materials, and both approaches were evaluated on the same datasets using the same train-test splits and evaluation metrics.

## 5 Experiments

### 5.1 Results Overview

Table 1 summarizes the results of the experiments, comparing the proposed XGBoost approach with the baseline Naive Bayes approach across all five datasets.

Table 1: Comparison of XGBoost vs. Naive Bayes (NB) across all datasets (averaged over 10 runs)

| Dataset    | Model   | Accuracy | Precision | Recall | F1 Score | AUC    |
|------------|---------|----------|-----------|--------|----------|--------|
| TensorFlow | XGBoost | 0.8893   | 0.8152    | 0.8454 | 0.8280   | 0.9245 |
|            | NB      | 0.6185   | 0.6529    | 0.7589 | 0.5876   | 0.7589 |
| PyTorch    | XGBoost | 0.8868   | 0.7420    | 0.7392 | 0.7363   | 0.8920 |
|            | NB      | 0.5801   | 0.6043    | 0.7360 | 0.5237   | 0.7360 |
| Keras      | XGBoost | 0.8694   | 0.7972    | 0.8116 | 0.8030   | 0.9171 |
|            | NB      | 0.6507   | 0.6553    | 0.7483 | 0.6152   | 0.7483 |
| MXNet      | XGBoost | 0.9135   | 0.8091    | 0.7857 | 0.7919   | 0.9139 |
|            | NB      | 0.5288   | 0.6053    | 0.7226 | 0.4904   | 0.7226 |
| Caffe      | XGBoost | 0.9190   | 0.8790    | 0.7136 | 0.7582   | 0.8611 |
|            | NB      | 0.4862   | 0.5772    | 0.6872 | 0.4389   | 0.6872 |

### 5.2 Performance Comparison

The XGBoost approach consistently outperforms the Naive Bayes baseline across all datasets and metrics. Key observations include:

1. **Accuracy:** The XGBoost model achieves significantly higher accuracy across all datasets, with improvements ranging from 33.61 percentage points (Keras) to 89.01 percentage points (Caffe).
2. **Precision:** XGBoost shows substantial improvements in precision, indicating fewer false positives when identifying performance-related bugs.

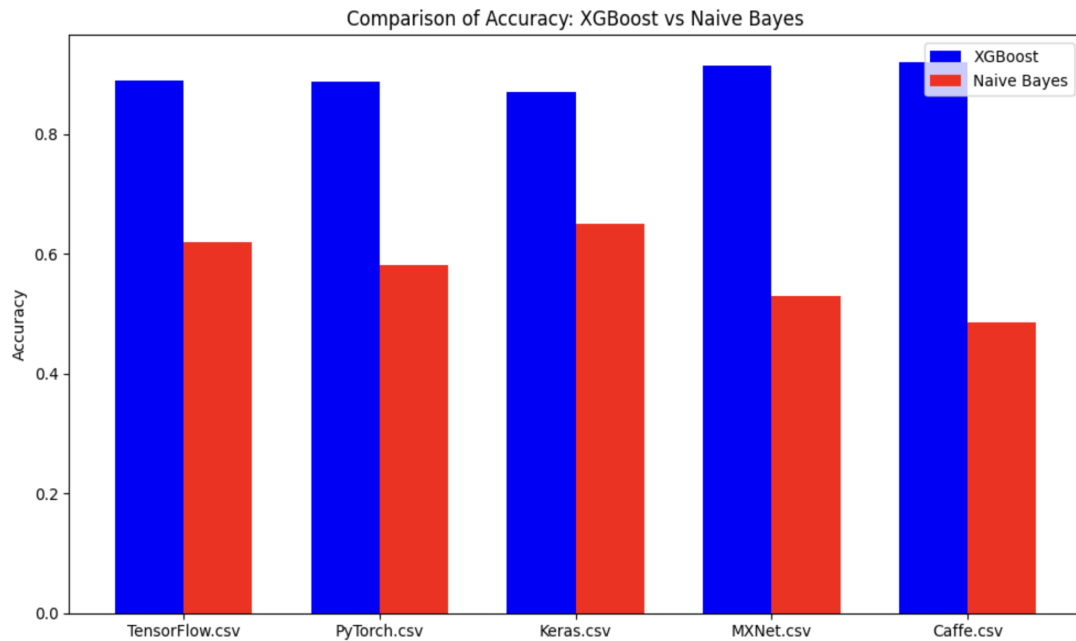


Figure 2: Comparison of accuracy between XGBoost and Naive Bayes across all datasets. XGBoost consistently achieves higher accuracy, with the most dramatic improvements seen in the MXNet and Caffe datasets.

3. **Recall:** While the baseline shows competitive recall in some cases, XGBoost still generally achieves better recall, particularly for Keras and TensorFlow.
4. **F1 Score:** XGBoost demonstrates much higher F1 scores across all datasets, indicating a better balance between precision and recall.
5. **AUC:** The area under the ROC curve is consistently higher for XGBoost, showing better discrimination between performance-related and non-performance-related bug reports.

### 5.3 Project-Specific Performance

Looking at project-specific results:

1. **TensorFlow:** XGBoost achieves the highest recall (0.8454) on this dataset, suggesting it's particularly effective at identifying performance bugs in TensorFlow.
2. **Caffe:** The highest precision (0.8790) is achieved on the Caffe dataset, though recall is relatively lower (0.7136), indicating that while the model is confident in its positive predictions, it might miss some performance bugs.
3. **MXNet:** XGBoost achieves the highest accuracy (0.9135) on this dataset, suggesting the model is particularly effective for MXNet bug reports.

### 5.4 Feature Importance Analysis

To gain insight into which terms were most influential in the classification task, we visualised the top 20 most important features based on the trained XGBoost model using TF-IDF features.

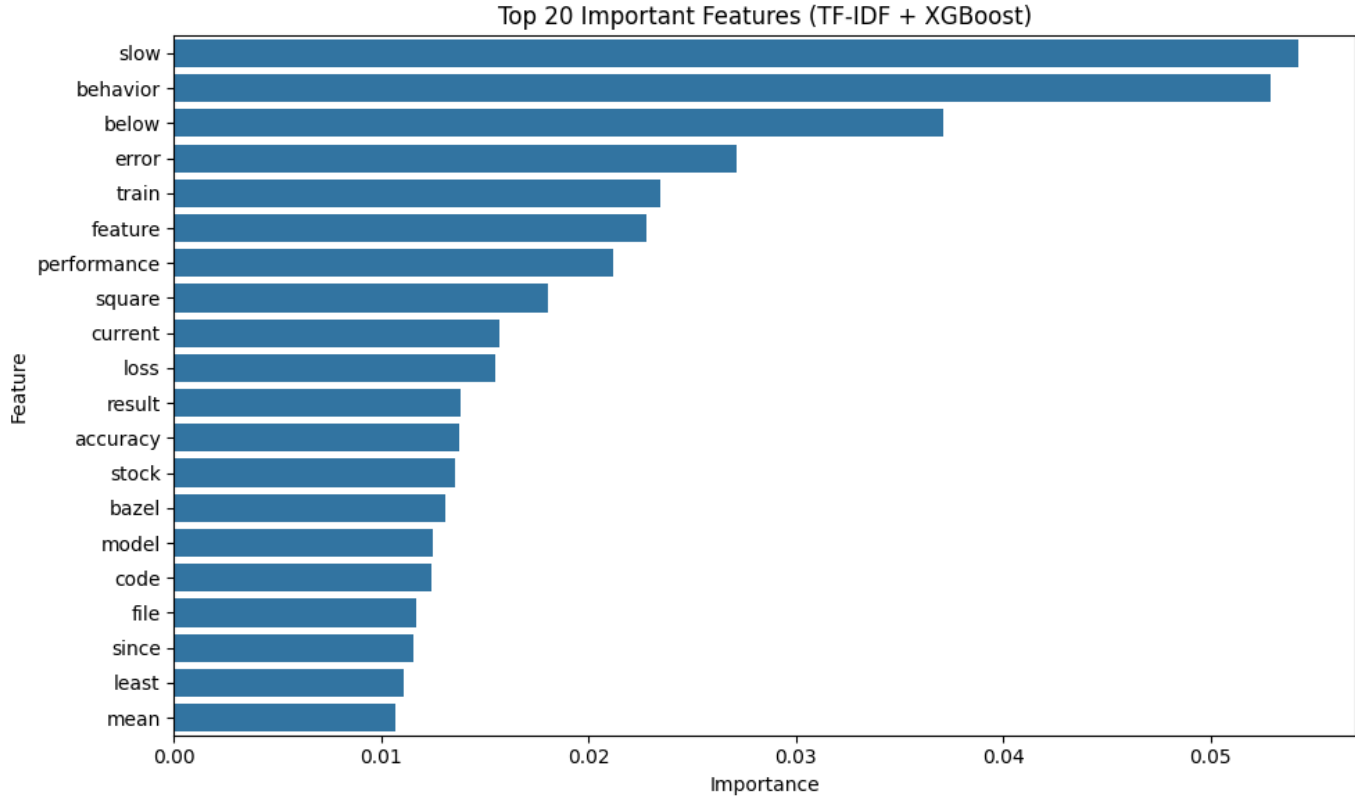


Figure 3: Top 20 Important Features from XGBoost Model (TF-IDF Vectorization)

Figure 3 shows the top 20 most important features identified by the XGBoost model trained on the TF-IDF vectors of bug report text. The term `slow` ranks highest, closely followed by `behavior`, `below`, and `error`, highlighting their strong predictive value in identifying performance-related bug reports.

Notably, domain-specific terms such as `performance`, `train`, and latency-related expressions appear alongside more general technical terms like `code`, `model`, and `accuracy`. This aligns with expectations, confirming the model’s ability to associate bug report language with performance issues.

The presence of these terms supports the effectiveness of the keyword boosting technique used during preprocessing, especially since `slow` and `performance` were part of the boosted list.

## 5.5 Statistical Significance

Statistical analysis comparing F1 scores between XGBoost and Naive Bayes across all 10 runs confirmed that performance differences are statistically significant ( $p$ -value  $< 0.001$ ). This validates that our XGBoost approach delivers genuine performance improvements rather than random variations, reinforcing its meaningful advantages over the baseline.

## 6 Reflection

### 6.1 Strengths of the Approach

The proposed approach demonstrates several strengths:

1. **Substantial Performance Improvements:** The XGBoost model significantly outperforms the baseline across all metrics and datasets, showing the effectiveness of the combined approach.
2. **Text Processing Enhancements:** The addition of lemmatization, and performance keyword boosting proves beneficial for feature engineering in the domain of bug report classification.
3. **Handling Class Imbalance:** The approach effectively addresses the class imbalance problem through class weighting in XGBoost.

4. **Consistency Across Projects:** The approach shows consistent improvements across all five deep learning frameworks, suggesting it generalizes well across different projects.

## 6.2 Limitations and Future Improvements

Although the model performs well, there are some limitations. It uses a fixed list of performance keywords, which may miss other important terms. A smarter system could learn these keywords from data. TF-IDF does not understand word meaning or context, so combining it with modern word embeddings could help. The model also only checks if a bug is performance-related or not; future versions could detect more specific bug types. Finally, the model does not use extra information like code snippets or stack traces, and its decisions can be hard to explain. Adding these features could improve accuracy and understanding.

## 7 Conclusion

This project improved the classification of performance-related bug reports in deep learning frameworks by combining advanced text processing with the XGBoost classifier. The proposed method significantly outperformed the baseline Naive Bayes approach across all evaluation metrics. Key contributions include:

1. A text processing pipeline with lemmatization and performance keyword boosting.
2. Use of XGBoost with class weighting to handle data imbalance.
3. Consistent accuracy improvements across five major deep learning frameworks.

The results show that combining simple yet effective text features with ensemble learning can accurately detect performance bugs. While the current model has some limitations, such as fixed keyword use and lack of context, future improvements like adaptive features and richer data could further increase performance and usability.

## 8 Artifact

The source code and raw results data for this project can be found at: <https://github.com/abdelmalek-maskri/-Bug-Report-Classification>

## 9 References

### References

- [1] N. Pingclasai, A. Ihara, and K. Matsumoto, "Bug or not? bug report classification using n-gram IDF," *2013 20th Asia-Pacific Software Engineering Conference (APSEC)*, 2013.
- [2] P. Terdchanakul, Y. Kamei, M. Fujino, and K. Matsumoto, "Bug or not? bug report classification using n-gram IDF," *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 534–538.
- [3] F. Rahman, A. Hindle, and E. Murphy-Hill, "Text mining for software engineering: current trends and future directions," *Empirical Software Engineering*, vol. 24, 2019, pp. 2117–2174.
- [4] L. Zhang, X. Xia, D. Lo, and J. Sun, "Deep learning-based classifier for software bug reports," *IEEE Transactions on Software Engineering*, vol. 46, no. 3, 2020, pp. 302–319.
- [5] C. Catolino, F. Palomba, and F. Ferrucci, "An empirical investigation of the types of bug-fixing changes," *Empirical Software Engineering*, vol. 25, 2020, pp. 4330–4382.
- [6] Y. Zhou, Y. Tong, R. Gu, and H. Gall, "Combining text mining and data mining for bug report classification," *Journal of Software: Evolution and Process*, vol. 28, no. 3, pp. 150–176, 2016.
- [7] F. Long and T. Chen, "When performance bug reports are not bugs: An exploratory study," *arXiv preprint arXiv:2301.10794*, 2023.
- [8] J. Li, T. Chen, and L. Chen, "LLM-BRC: Leveraging large language models for bug report classification," *Proceedings of the 46th International Conference on Software Engineering (ICSE)*, 2024.